

Name:K.Srija

Roll No:2303A52189

1. Print the boundary nodes of a binary tree.

```
class Solution {
    ArrayList<Integer> boundaryTraversal(Node root) {
        ArrayList<Integer> ans = new ArrayList<>();
        if (root == null) {
            return ans;
        }
        if (!isLeaf(root)) {
            ans.add(root.data);
            Node curr = root.left;
            while (curr != null) {
                if (!isLeaf(curr)) {
                    ans.add(curr.data);
                }
                if (curr.left != null) {
                    curr = curr.left;
                } else {
                    curr = curr.right;
                }
            }
            addNodes(root, ans);
            ArrayList<Integer> right = new ArrayList<>();
            curr = root.right;
            while (curr != null) {
                if (!isLeaf(curr)) {
                    right.add(curr.data);
                }
                if (curr.right != null) {
                    curr = curr.right;
                } else {
                    curr = curr.left;
                }
            }
            for (int i = right.size() - 1; i >= 0; i--) {
                ans.add(right.get(i));
            }
        }
    }

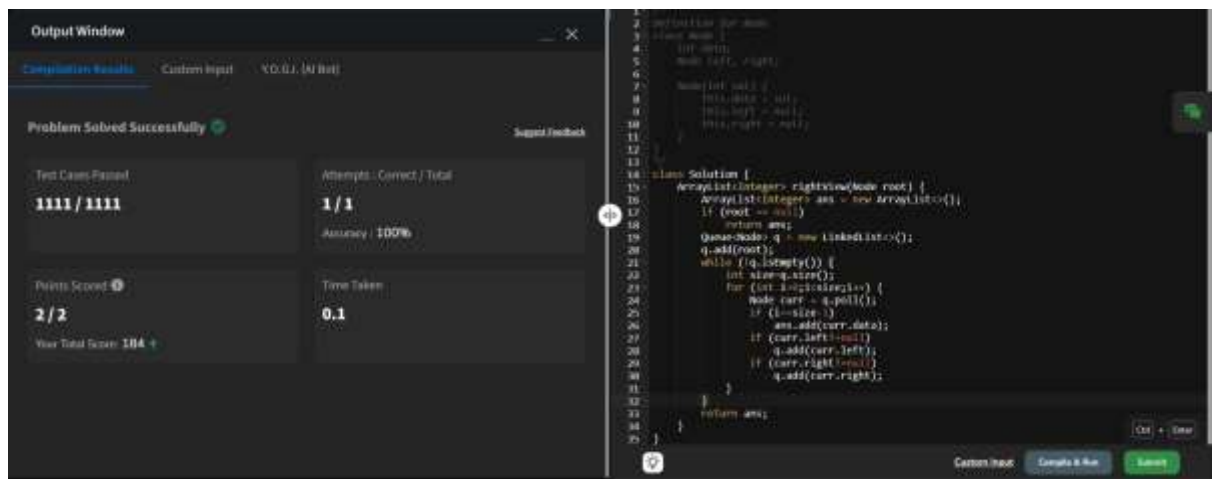
    boolean isLeaf(Node node) {
        return node.left == null && node.right == null;
    }

    void addNodes(Node root, ArrayList<Integer> ans) {
        if (root == null) {
            return;
        }
        if (!isLeaf(root)) {
            ans.add(root.data);
        }
        addNodes(root.left, ans);
        addNodes(root.right, ans);
    }
}
```

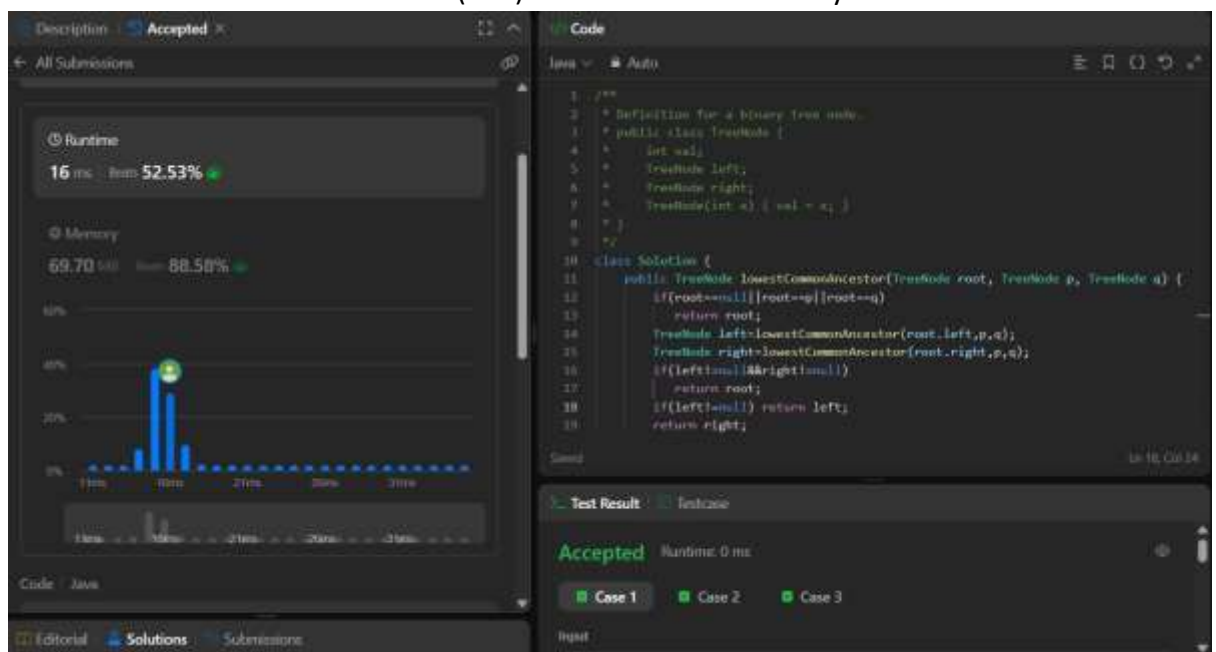
2. Print the left view of a binary tree.

```
class Solution {
    ArrayList<Integer> leftView(Node root) {
        ArrayList<Integer> ans = new ArrayList<>();
        if (root == null) {
            return ans;
        }
        Queue<Node> q = new LinkedList<>();
        q.add(root);
        while (!q.isEmpty()) {
            int size = q.size();
            for (int i = 0; i < size; i++) {
                Node curr = q.poll();
                if (i == 0) {
                    ans.add(curr.data);
                }
                if (curr.left != null) {
                    q.add(curr.left);
                }
                if (curr.right != null) {
                    q.add(curr.right);
                }
            }
        }
        return ans;
    }
}
```

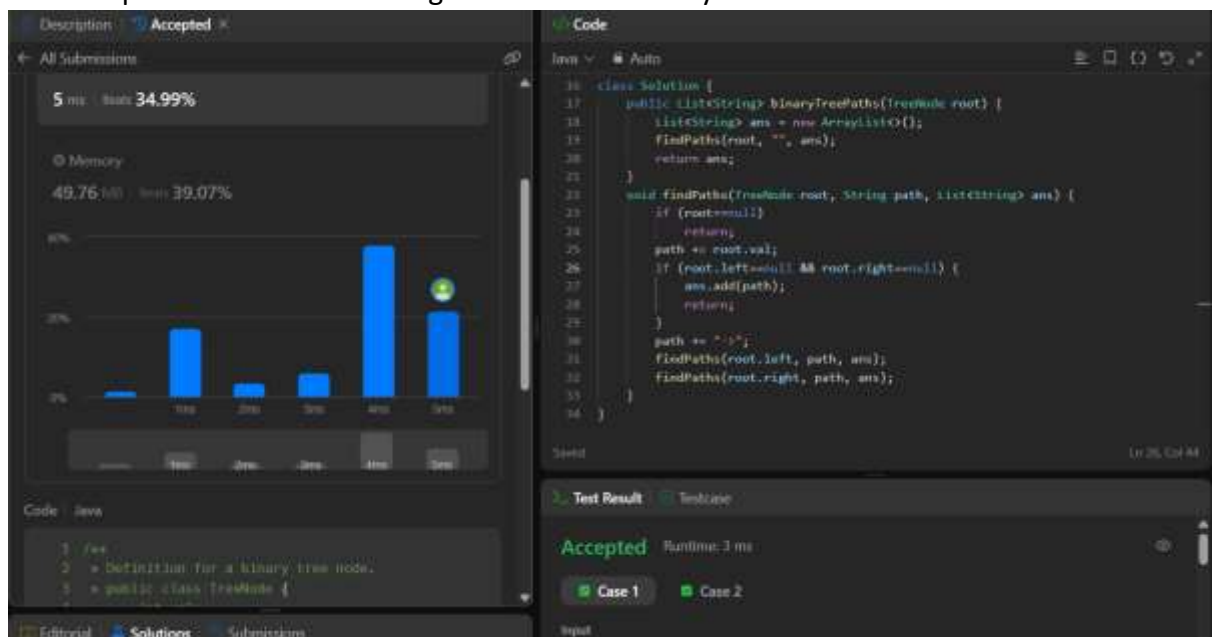
3. Print the right view of a binary tree.



4. Find the lowest common ancestor (LCA) of two nodes in a binary tree.



5. Find the path from the root to a given node in a binary tree.



6. Construct a binary tree from inorder and preorder traversal.

The screenshot displays a code editor interface for a Java solution. On the left, a 'Description' tab shows 'Accepted' status. Below it, a 'Runtime' summary indicates 3 ms and 29.51% memory usage. A 'Memory' section shows 45.28 MB and 93.15% usage. A bar chart visualizes memory usage across different test cases. The 'Code' tab on the right contains the following Java code:

```
1 class Solution {
2     public TreeNode buildTree(int[] preorder, int[] inorder) {
3         return build(preorder, inorder, 0, 0, inorder.length - 1);
4     }
5     int preIndex = 0;
6     TreeNode build(int[] preorder, int[] inorder, int start, int left, int right)
7     {
8         if (left > right)
9             return null;
10        TreeNode root = new TreeNode(preorder[preIndex++]);
11        int index = left;
12        while (inorder[index] != root.val)
13            index++;
14        root.left = build(preorder, inorder, start, left, index - 1);
15        root.right = build(preorder, inorder, start, index + 1, right);
16        return root;
17    }
18 }
```

Below the code, the 'Test Result' tab shows 'Accepted' status with a runtime of 0 ms. Two test cases, 'Case 1' and 'Case 2', are listed as passed. The bottom of the interface shows tabs for 'Editorial', 'Solutions', and 'Submissions'.